THE UNIVERSITY OF MELBOURNE

COMP90038 Algorithms and Complexity

Lecture 10: Decrease-and-Conquer-by-a-Factor

Olya Ohrimenko
(with thanks to Harald Søndergaard)

Decrease-and-Conquer

- **Previous lecture:** to solve a problem of size n, try to express the solution in terms of a solution to the same problem of size n−1.

- A simple example was sorting: To sort an array of length n, just:
  1. sort the first n − 1 items, then
  2. locate the cell A[j] that should hold the last item, right-shift all elements to its right, then place the last element in A[j].

- This led to an $O(n^2)$ algorithm called **insertion sort**. We can implement the idea either with recursion or iteration (we chose iteration).

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

What input is worst&best for insertion sort?



THE UNIVERSITY OF MELBOURNE

a

9	12	23	41	52	46	83

b

83	52	46	41	23	12	9

function INSERTIONSORT($A[\cdot], n$)
  for $i \leftarrow 1$ **to** $n-1$ **do**
    $v \leftarrow A[i]$
    $j \leftarrow i-1$
    while $j \geq 0$ **and** $v < A[j]$ **do**
      $A[j+1] \leftarrow A[j]$
      $j \leftarrow j-1$
    $A[j+1] \leftarrow v$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Decrease-and-Conquer by-a-Factor

THE UNIVERSITY OF MELBOURNE

- We now look at better utilization of the approach, often leading to methods with logarithmic time behaviour or better!

- **Decrease-by-a-constant-factor** is exemplified by binary search.

- **Decrease-by-a-variable-factor** is exemplified by interpolation search.

- Let us look at these and other instances.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Left blank

THE UNIVERSITY OF MELBOURNE

$$A[0] \quad \cdots \quad A[m-1] \quad A[m] \quad A[m+1] \quad \cdots \quad A[n-1]$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

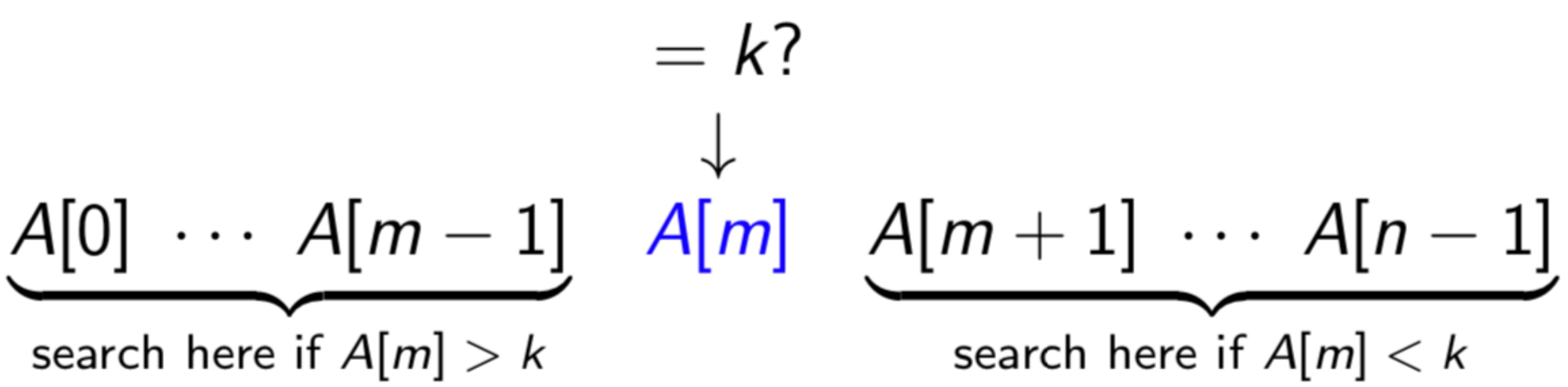
Binary Search

- This is a well-known approach for searching for an element ***k*** in a sorted array.

- Start by comparing against the array's middle element A[***m***]. If A[***m***] = ***k*** we are done.

- If A[m] > k, search the sub-array up to A[m – 1] recursively.

- If A[m] < k, search the sub-array from A[m + 1] recursively.



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Binary Search

We have already seen a recursive formulation in Lecture 4. Here is an iterative one.

```
function BinSearch(A[·], n, k)
    lo ← 0
    hi ← n − 1
    while lo ≤ hi do
        m ← ⌊(lo + hi)/2⌋
        if A[m] = k then
            return m
        if A[m] > k then
            hi ← m − 1
        else
            lo ← m + 1
    return −1
```


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example:
Binary Search in Sorted Array

```
function BinSearch(A[·], n, k)
    lo ← 0
    hi ← n − 1
    while lo ≤ hi do
        m ← ⌊(lo + hi)/2⌋
        if A[m] = k then
            return m
        if A[m] > k then
            hi ← m − 1
        else
            lo ← m + 1
    return −1
```

k: 41

lo: 0

hi: 6

m: 3

A:

4	9	13	22	41	83	96
0	1	2	3	4	5	6

BinSearch(A,7,41)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example:
Binary Search in Sorted Array

```
function BinSearch(A[·], n, k)
    lo ← 0
    hi ← n − 1
    while lo ≤ hi do
        m ← ⌊(lo + hi)/2⌋
        if A[m] = k then
            return m
        if A[m] > k then
            hi ← m − 1
        else
            lo ← m + 1
    return −1
```

k: 41

lo: 4

hi: 6

m: 3

A:

4	9	13	22	41	83	96
0	1	2	3	4	5	6

BinSearch(A,7,41)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example:
Binary Search in Sorted Array

THE UNIVERSITY OF MELBOURNE

```
function BinSearch(A[·], n, k)
    lo ← 0
    hi ← n − 1
    while lo ≤ hi do
        m ← ⌊(lo + hi)/2⌋
        if A[m] = k then
            return m
        if A[m] > k then
            hi ← m − 1
        else
            lo ← m + 1
    return −1
```

k: 41

lo: 4

hi: 6

m: 5

A:

0	1	2	3	4	5	6
4	9	13	22	41	83	96

BinSearch(A,7,41)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example:
Binary Search in Sorted Array

THE UNIVERSITY OF MELBOURNE

```
function BinSearch(A[·], n, k)
    lo ← 0
    hi ← n − 1
    while lo ≤ hi do
        m ← ⌊(lo + hi)/2⌋
        if A[m] = k then
            return m
        if A[m] > k then
            hi ← m − 1
        else
            lo ← m + 1
    return −1
```

k: 41

lo: 4

hi: 4

m: 5

A:

4	9	13	22	41	83	96
0	1	2	3	4	5	6

BinSearch(A,7,41)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example:
Binary Search in Sorted Array

THE UNIVERSITY OF MELBOURNE

```
function BinSearch(A[·], n, k)
    lo ← 0
    hi ← n − 1
    while lo ≤ hi do
        m ← ⌊(lo + hi)/2⌋
        if A[m] = k then
            return m
        if A[m] > k then
            hi ← m − 1
        else
            lo ← m + 1
    return −1
```

k: 41

lo: 4

hi: 4

m: 4

A:

4	9	13	22	41	83	96
0	1	2	3	4	5	6

BinSearch(A,7,41)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Complexity of Binary Search



THE UNIVERSITY OF MELBOURNE

- Worst-case input to binary sarch:
  - When ***k*** is not in the array

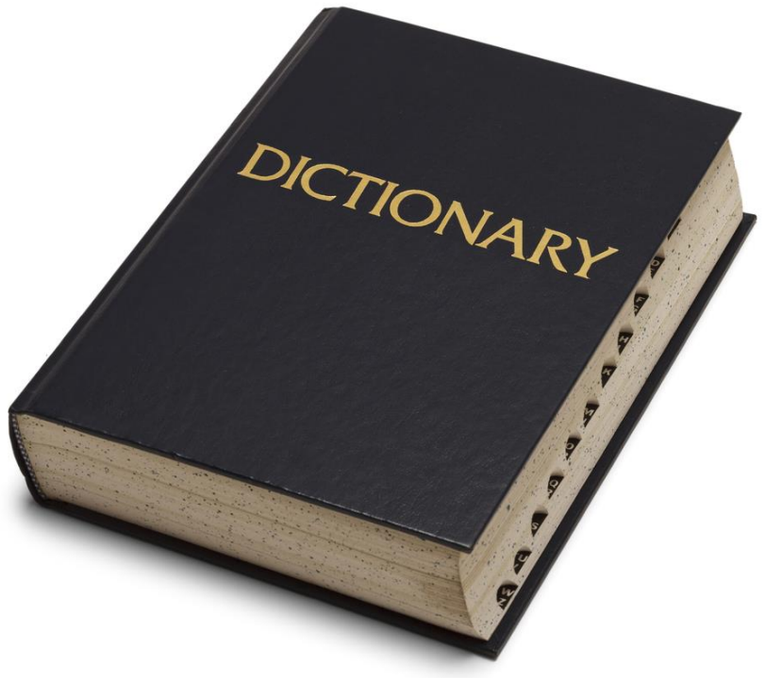
- In that case, its complexity is given by the following recursive equation:

$$C(n) = \begin{cases} 1 & \text{if } n = 1 \\ C(\lfloor n/2 \rfloor) + 1 & \text{if } n > 1 \end{cases}$$

- A closed form is: $C(n) = \lfloor \log_2 n \rfloor + 1$
- In the worst case, searching for k in an array of size 1,000,000 requires 20 comparisons.
- The average-case time complexity is also Θ(log n)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Interpolation Search



https://www.collinsdictionary.com/images/full/dictionary_168552845.jpg

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Interpolation Search

- If the elements of a sorted array are distributed reasonably evenly, we can do better than binary search!

- Think about how you search for an entry in the telephone directory: If you look for ‘Zobel’, you make a rough estimate of where to do the first probe—very close to the end of the directory.

- This is the idea in interpolation search.

- When searching for k in the array segment A[lo] to A[hi], take into account where k is, relative to A[lo] and A[hi].

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Interpolation Search



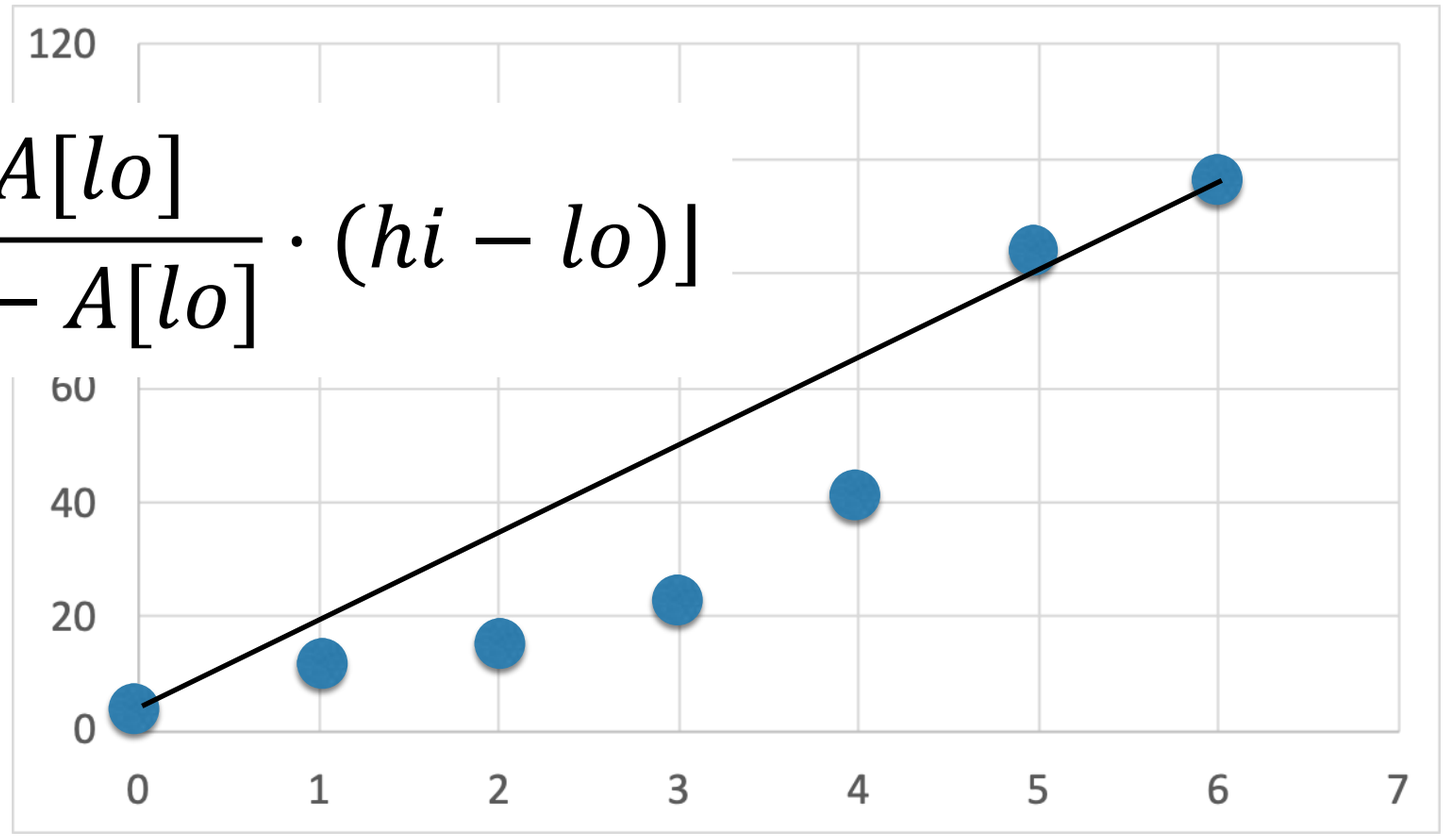
THE UNIVERSITY OF MELBOURNE

A:

4	9	13	22	41	83	96
0	1	2	3	4	5	6

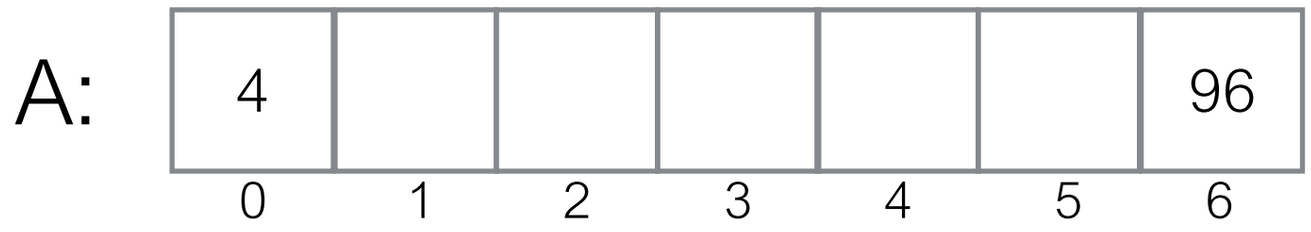
Suppose we are searching for k = 83

$$mid = lo + \lfloor \frac{k - A[lo]}{A[hi] - A[lo]} \cdot (hi - lo) \rfloor$$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

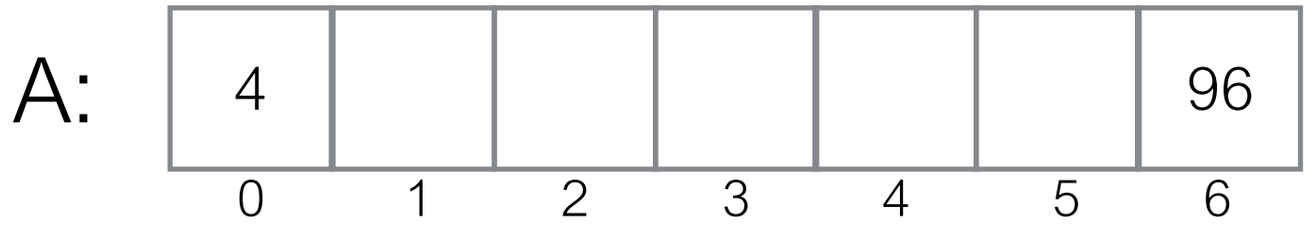
Intuition



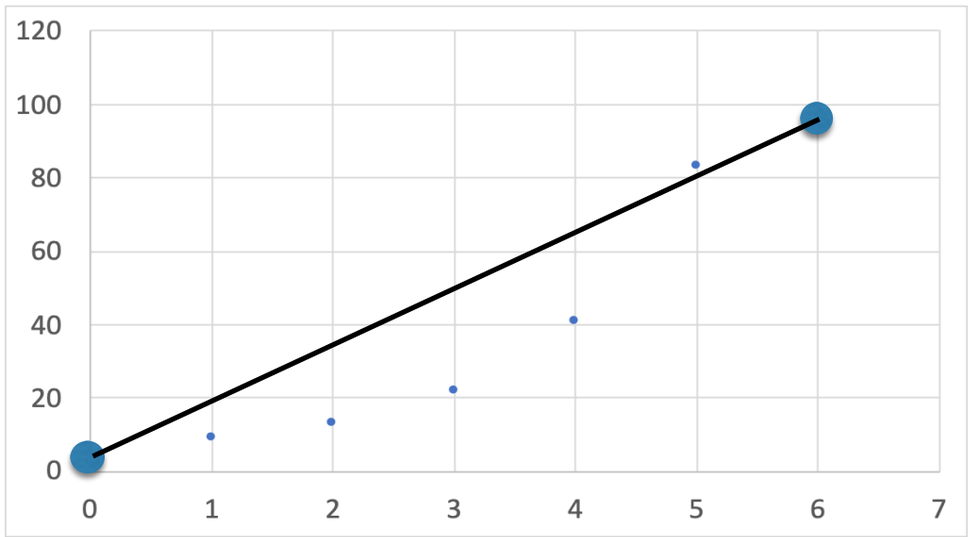
Suppose we are searching for k = 83

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Intuition



Suppose we are searching for k = 83



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Interpolation Search

- Instead of computing the mid-point m as in binary search:

$$m \leftarrow \lfloor (lo + hi)/2 \rfloor$$

we instead perform linear interpolation between the points (lo,A[lo]) and (hi,A[hi]). That is, we use:

$$m \leftarrow lo + \lfloor \frac{k - A[lo]}{A[hi] - A[lo]} (hi - lo) \rfloor$$

- Interpolation search has average complexity O(log log n)

- It is the right choice for large arrays when elements are **uniformly distributed**

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Finding the Median

- Given an array, an important problem is how to find the **median**, that is, an array value which is no larger than half the elements and no smaller than half.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Finding the Median

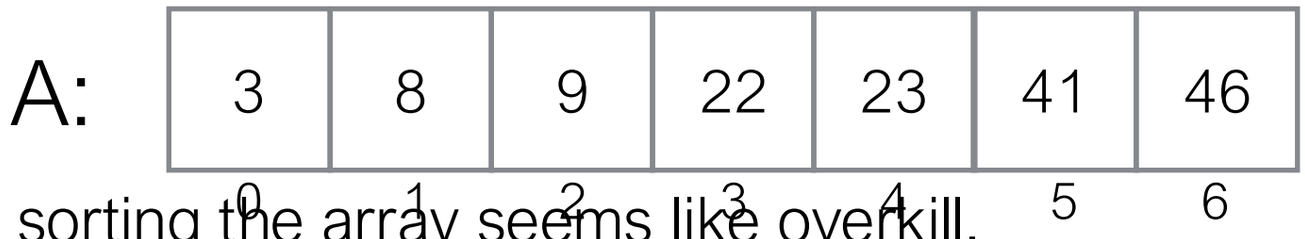
- Given an array, an important problem is how to find the **median**, that is, an array value which is no larger than half the elements and no smaller than half.



- More generally, we would like to solve the problem of finding the kth smallest element. (e.g. when k=3)



- If the array is sorted, the solution is straight-forward, so one approach is to start by sorting (as we'll soon see, this can be done in time O (n log n)).



- However, sorting the array seems like overkill.

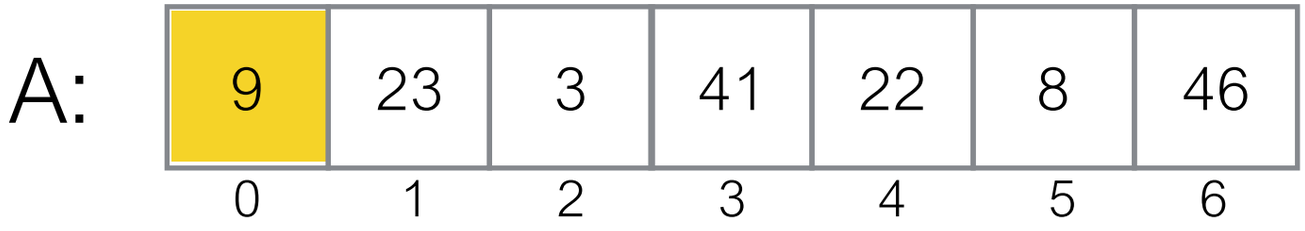
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Left blank

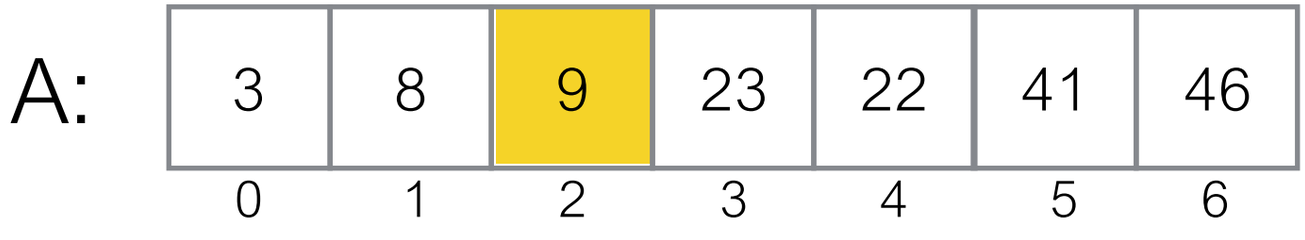
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

A Detour via Partitioning

- Partitioning an array around some pivot element p means reorganizing the array so that all elements to the left of p are no greater than p, while those to the right are no smaller.

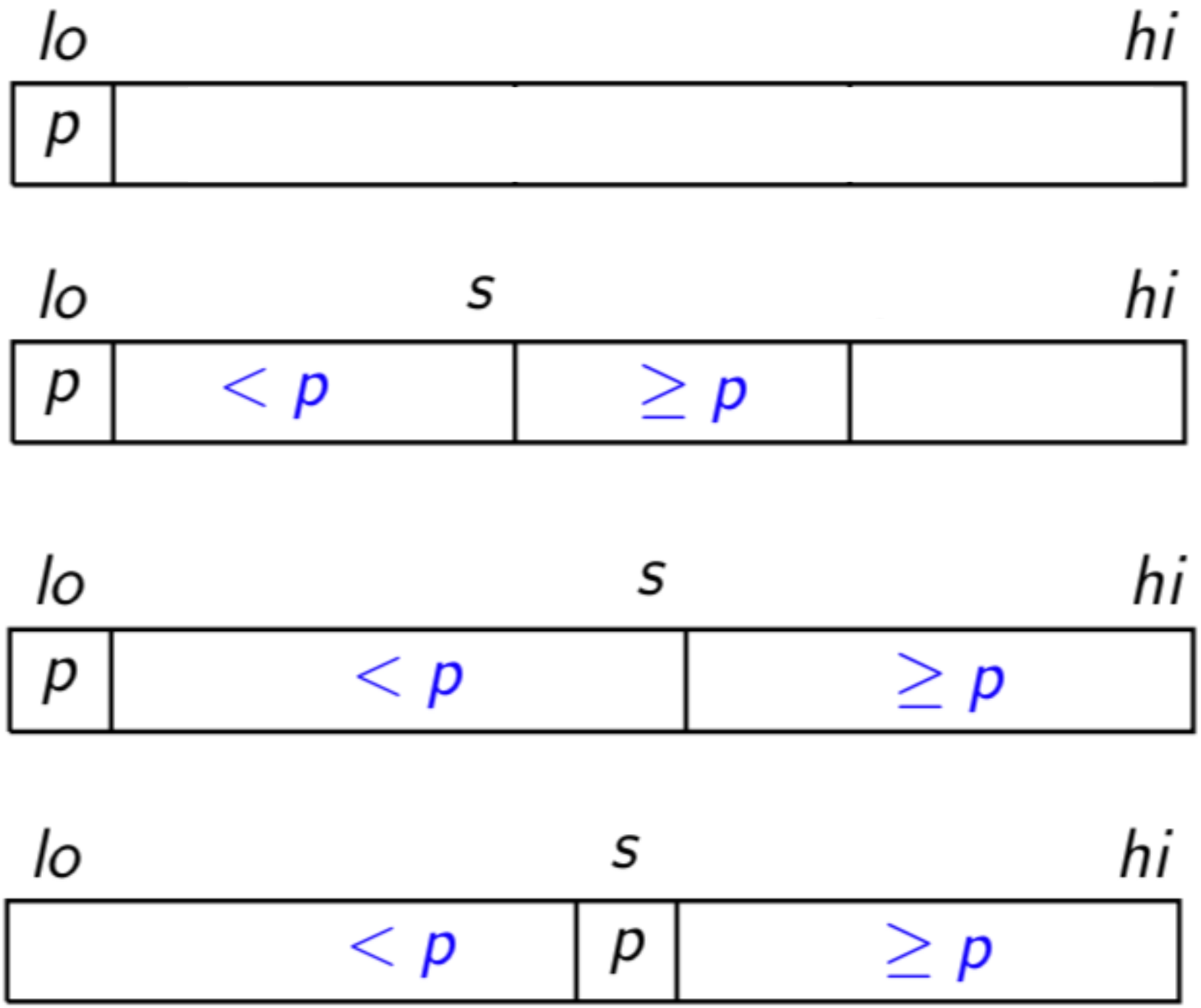


Partitioning around the pivot 9



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Lomuto Partitioning

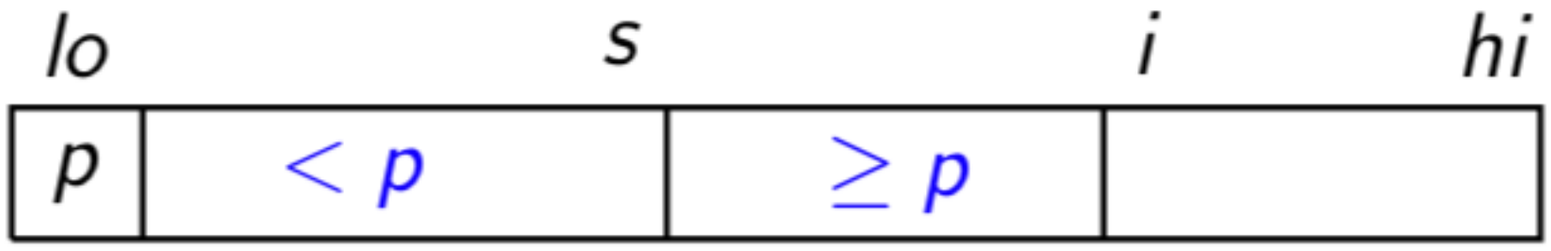


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Lomuto Partitioning

THE UNIVERSITY OF MELBOURNE

```
function LomutoPartition(A[·], lo, hi)
    p ← A[lo]
    s ← lo
    for i ← lo + 1 to hi do
        if A[i] < p then
            s ← s + 1
            swap(A[s], A[i])
    swap(A[lo], A[s])
    return s
```

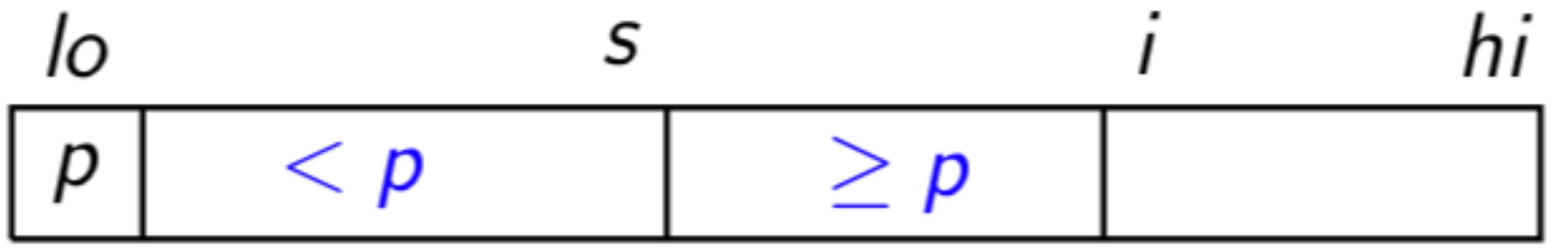


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Lomuto Partitioning

```
function LomutoPartition(A[·], lo, hi)
    p ← A[lo]
    s ← lo
    for i ← lo + 1 to hi do
        if A[i] < p then
            s ← s + 1
            swap(A[s], A[i])
    swap(A[lo], A[s])
    return s
```





Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Lomuto Partitioning



THE UNIVERSITY OF MELBOURNE

function LomutoPartition($A[\cdot], lo, hi$)

$p \leftarrow A[lo]$

$s \leftarrow lo$

for $i \leftarrow lo + 1$ **to** hi **do**

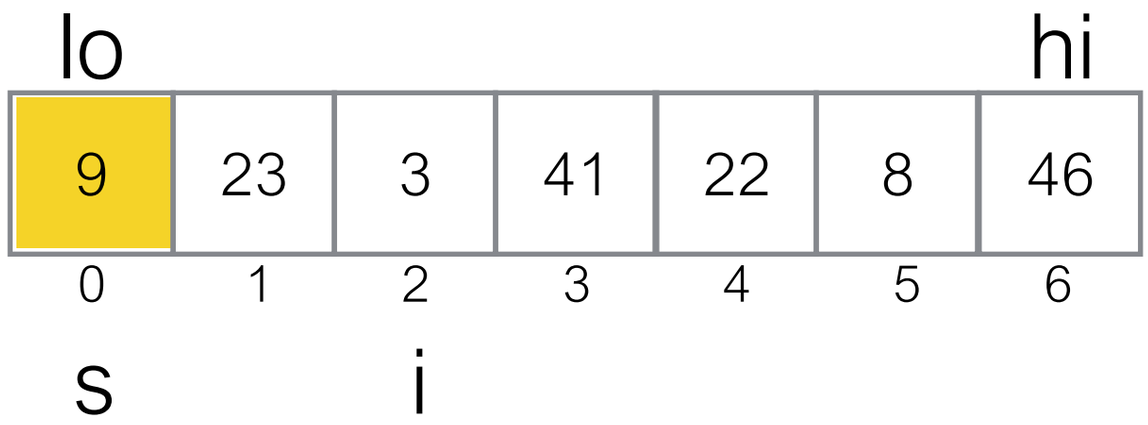
if $A[i] < p$ **then**

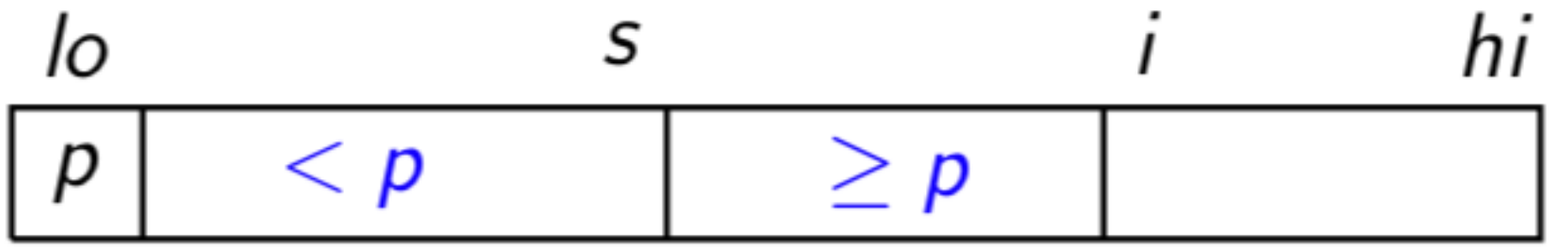
$s \leftarrow s + 1$

$swap(A[s], A[i])$

$swap(A[lo], A[s])$

return s





Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Lomuto Partitioning



THE UNIVERSITY OF MELBOURNE

function LomutoPartition($A[\cdot], lo, hi$)

$p \leftarrow A[lo]$

$s \leftarrow lo$

for $i \leftarrow lo + 1$ **to** hi **do**

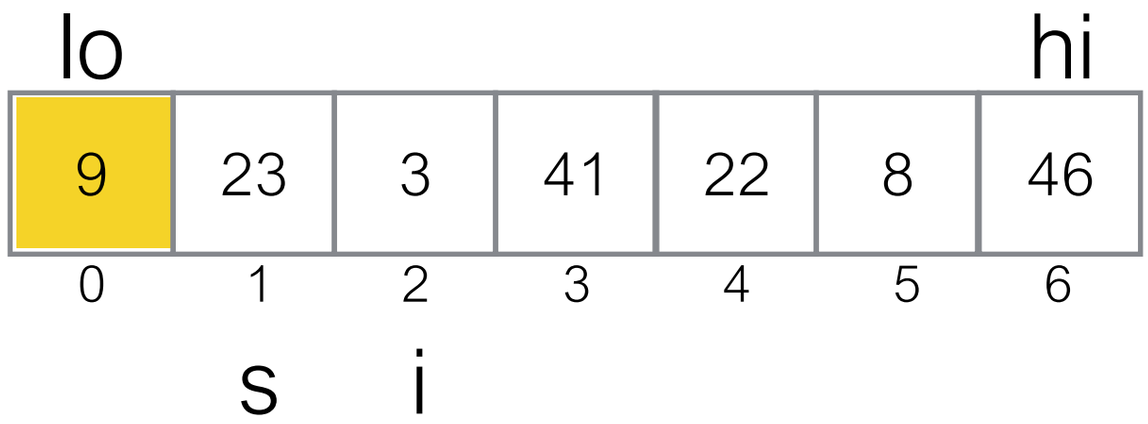
if $A[i] < p$ **then**

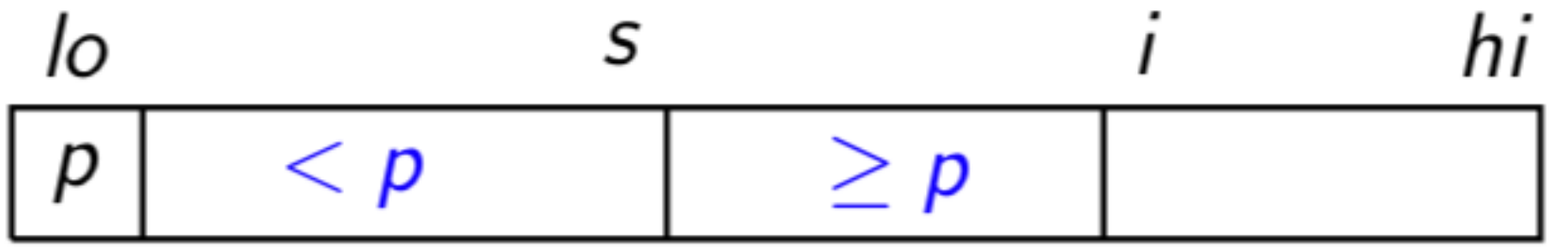
$s \leftarrow s + 1$

$swap(A[s], A[i])$

$swap(A[lo], A[s])$

return s





Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Lomuto Partitioning



THE UNIVERSITY OF MELBOURNE

function LomutoPartition($A[\cdot], lo, hi$)

$p \leftarrow A[lo]$

$s \leftarrow lo$

for $i \leftarrow lo + 1$ **to** hi **do**

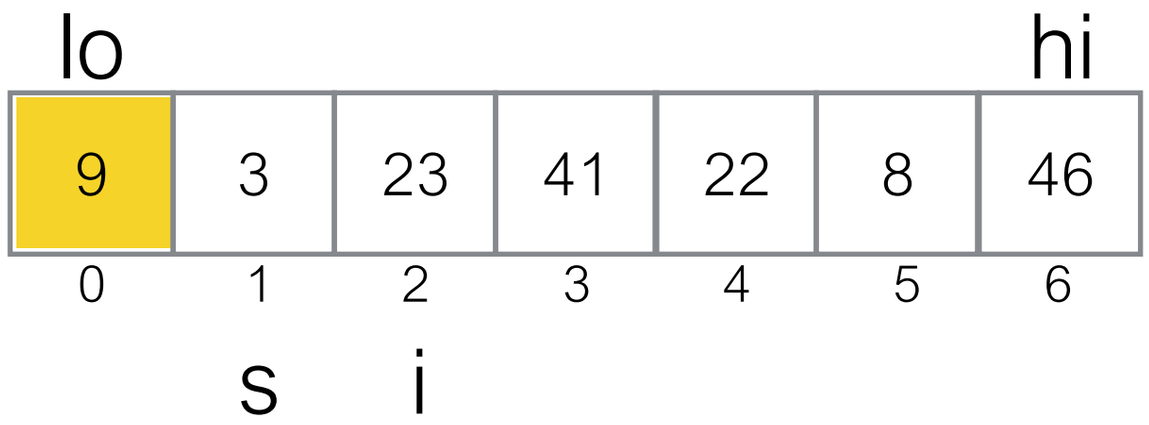
if $A[i] < p$ **then**

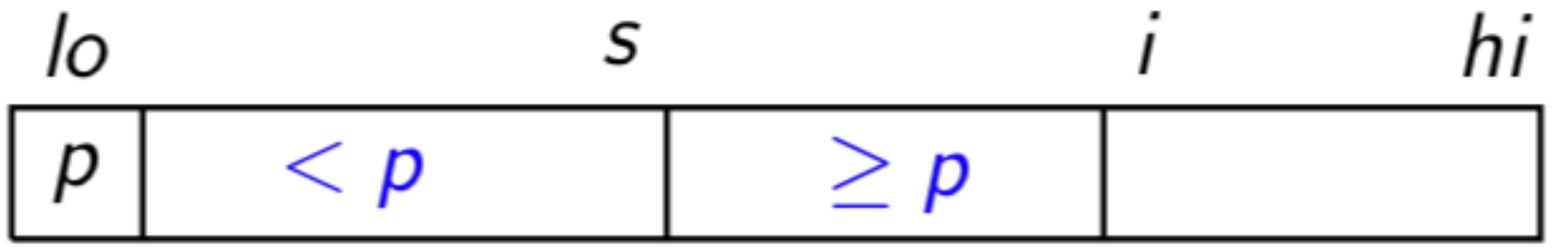
$s \leftarrow s + 1$

$swap(A[s], A[i])$

$swap(A[lo], A[s])$

return s





Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Lomuto Partitioning



THE UNIVERSITY OF MELBOURNE

function LomutoPartition($A[\cdot], lo, hi$)

$p \leftarrow A[lo]$

$s \leftarrow lo$

for $i \leftarrow lo + 1$ **to** hi **do**

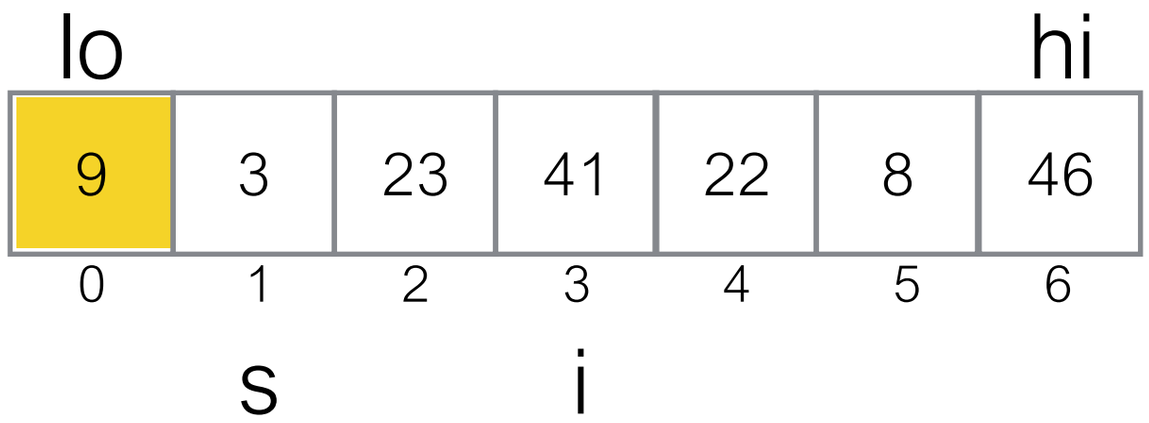
if $A[i] < p$ **then**

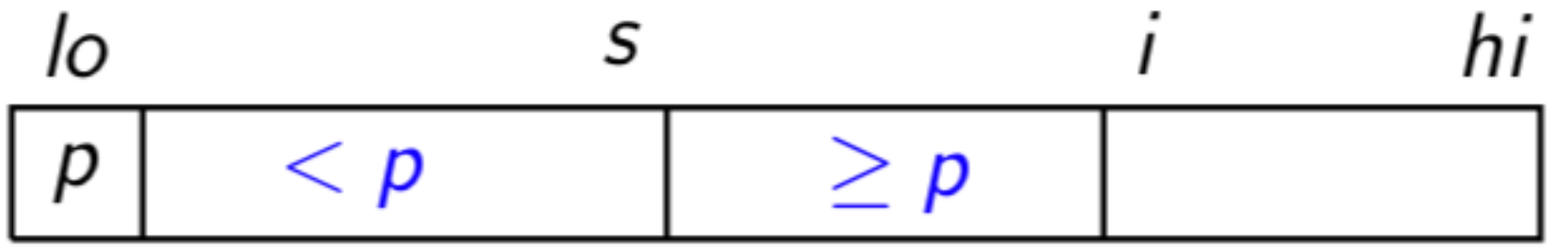
$s \leftarrow s + 1$

$swap(A[s], A[i])$

$swap(A[lo], A[s])$

return s

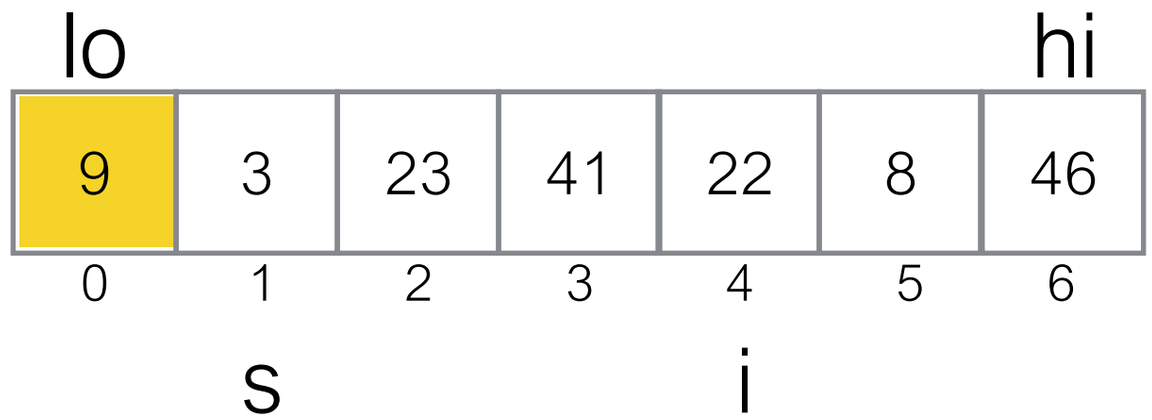


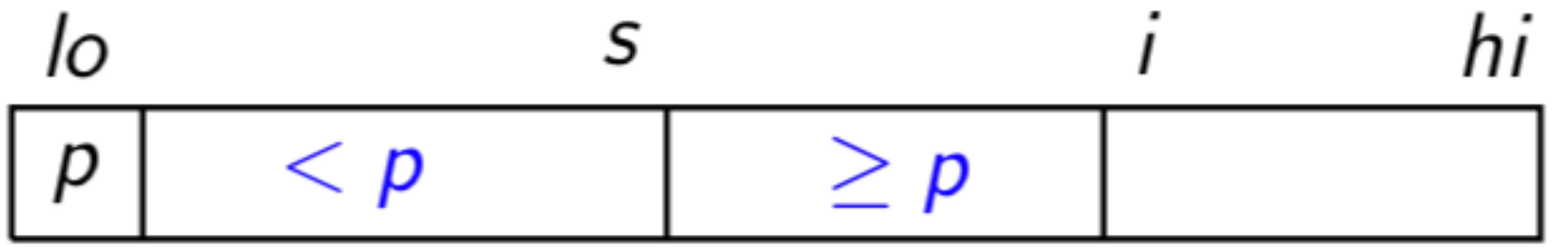


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Lomuto Partitioning

```
function LomutoPartition(A[·], lo, hi)
    p ← A[lo]
    s ← lo
    for i ← lo + 1 to hi do
        if A[i] < p then
            s ← s + 1
            swap(A[s], A[i])
    swap(A[lo], A[s])
    return s
```





Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Lomuto Partitioning



THE UNIVERSITY OF MELBOURNE

function LomutoPartition($A[\cdot], lo, hi$)

$p \leftarrow A[lo]$

$s \leftarrow lo$

for $i \leftarrow lo + 1$ **to** hi **do**

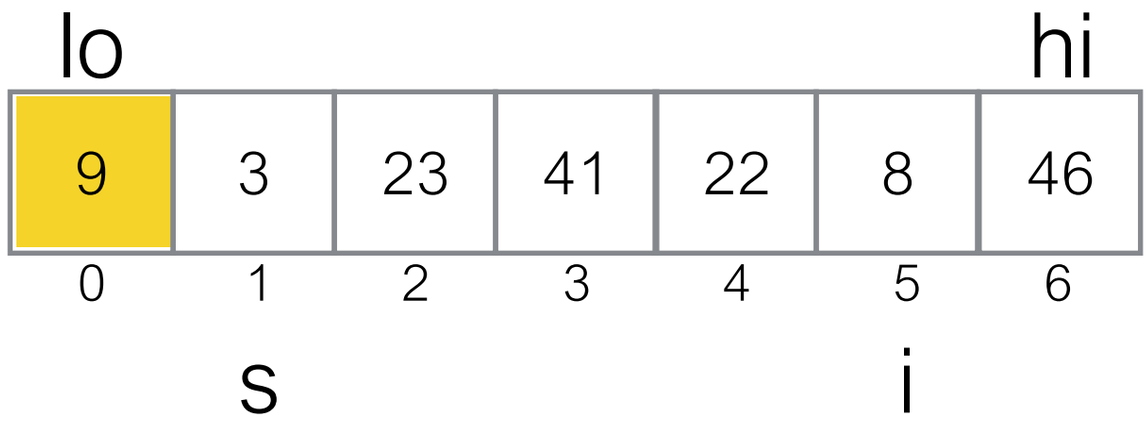
if $A[i] < p$ **then**

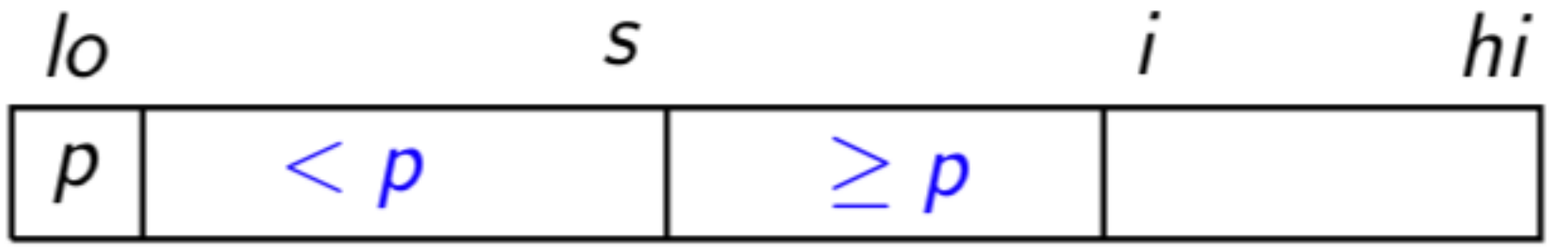
$s \leftarrow s + 1$

$swap(A[s], A[i])$

$swap(A[lo], A[s])$

return s

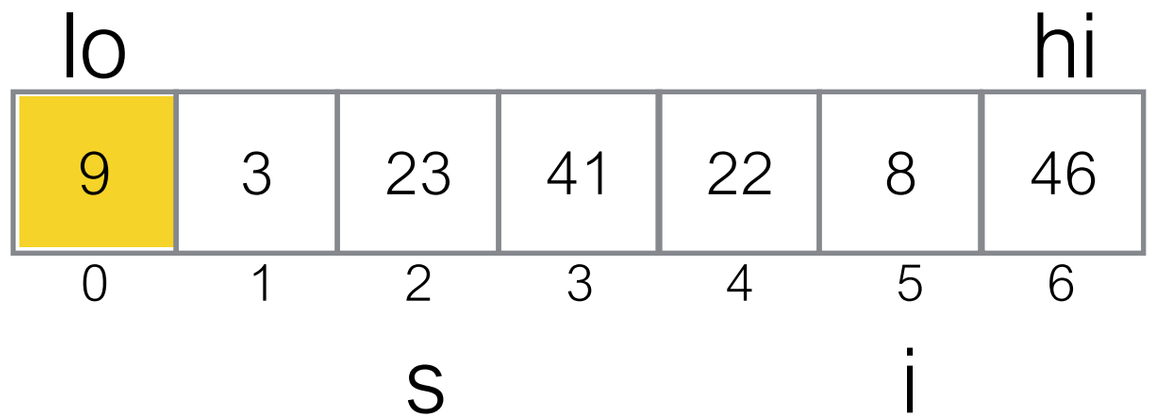


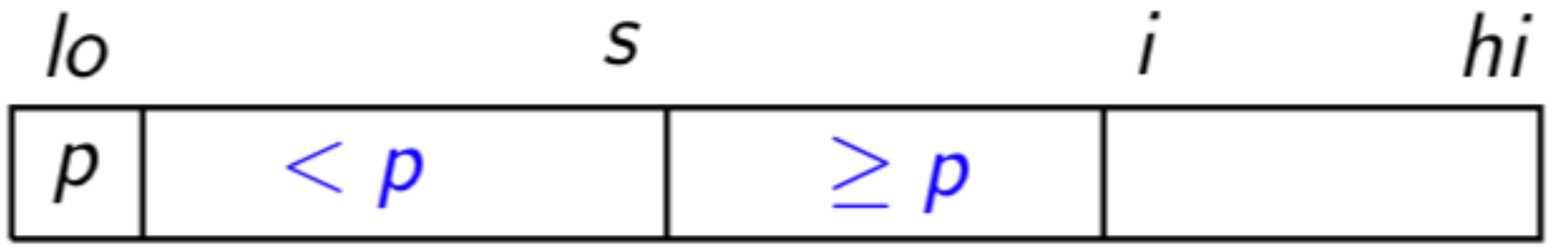


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Lomuto Partitioning

```
function LomutoPartition(A[·], lo, hi)
    p ← A[lo]
    s ← lo
    for i ← lo + 1 to hi do
        if A[i] < p then
            s ← s + 1
            swap(A[s], A[i])
    swap(A[lo], A[s])
    return s
```





Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Lomuto Partitioning



THE UNIVERSITY OF MELBOURNE

function LomutoPartition($A[\cdot], lo, hi$)

$p \leftarrow A[lo]$

$s \leftarrow lo$

for $i \leftarrow lo + 1$ **to** hi **do**

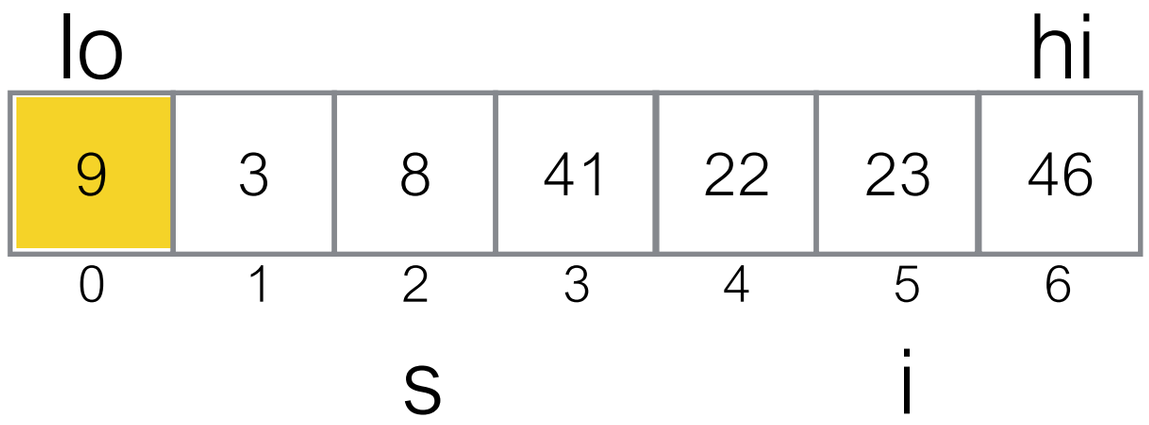
if $A[i] < p$ **then**

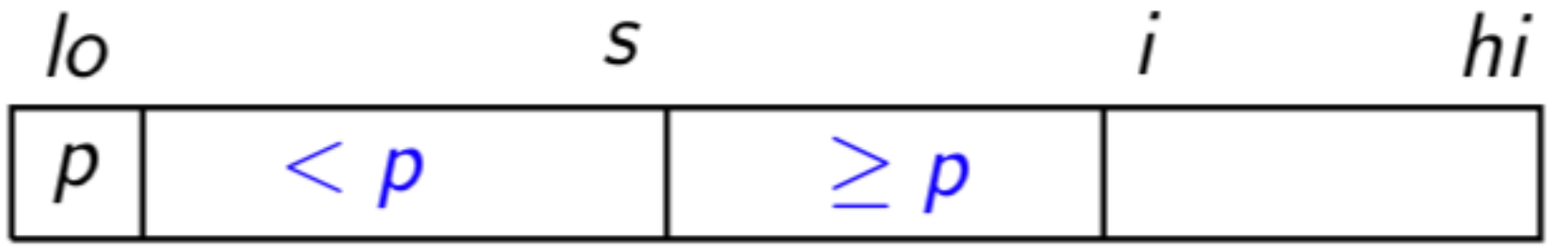
$s \leftarrow s + 1$

$swap(A[s], A[i])$

$swap(A[lo], A[s])$

return s





Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Lomuto Partitioning



THE UNIVERSITY OF MELBOURNE

function LomutoPartition($A[\cdot], lo, hi$)

$p \leftarrow A[lo]$

$s \leftarrow lo$

for $i \leftarrow lo + 1$ **to** hi **do**

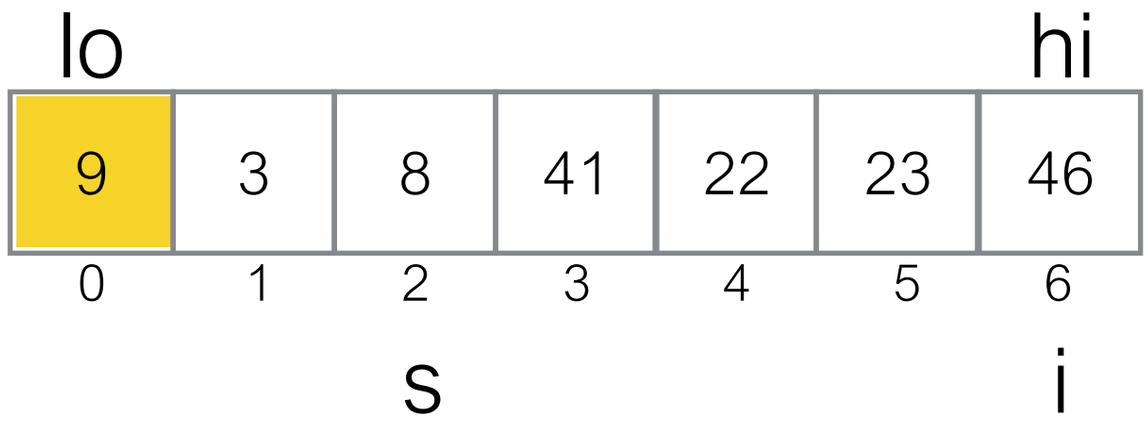
if $A[i] < p$ **then**

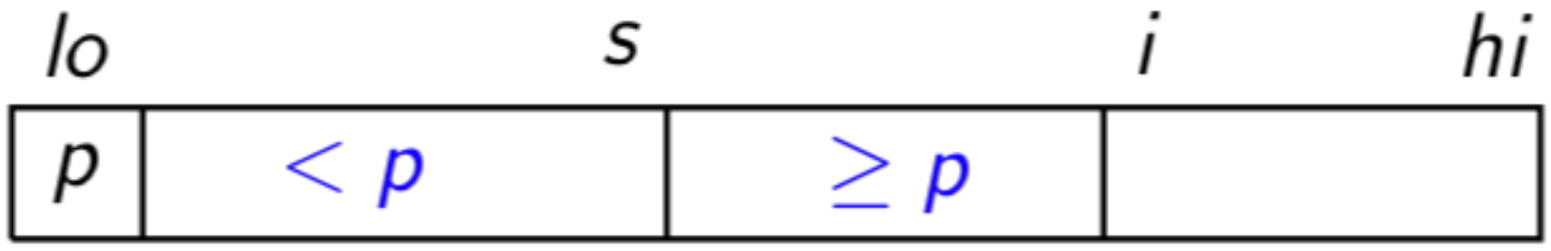
$s \leftarrow s + 1$

$swap(A[s], A[i])$

$swap(A[lo], A[s])$

return s





Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Lomuto Partitioning



THE UNIVERSITY OF MELBOURNE

function LomutoPartition($A[\cdot], lo, hi$)

$p \leftarrow A[lo]$

$s \leftarrow lo$

for $i \leftarrow lo + 1$ **to** hi **do**

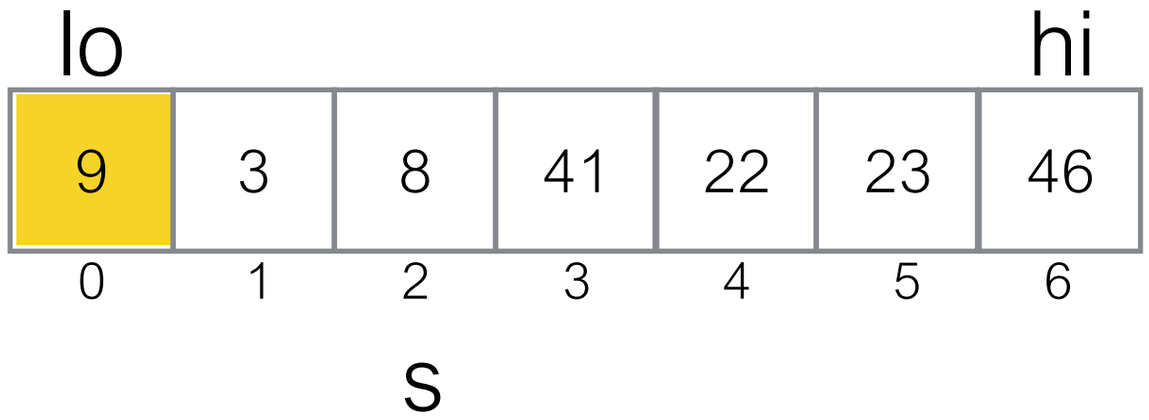
if $A[i] < p$ **then**

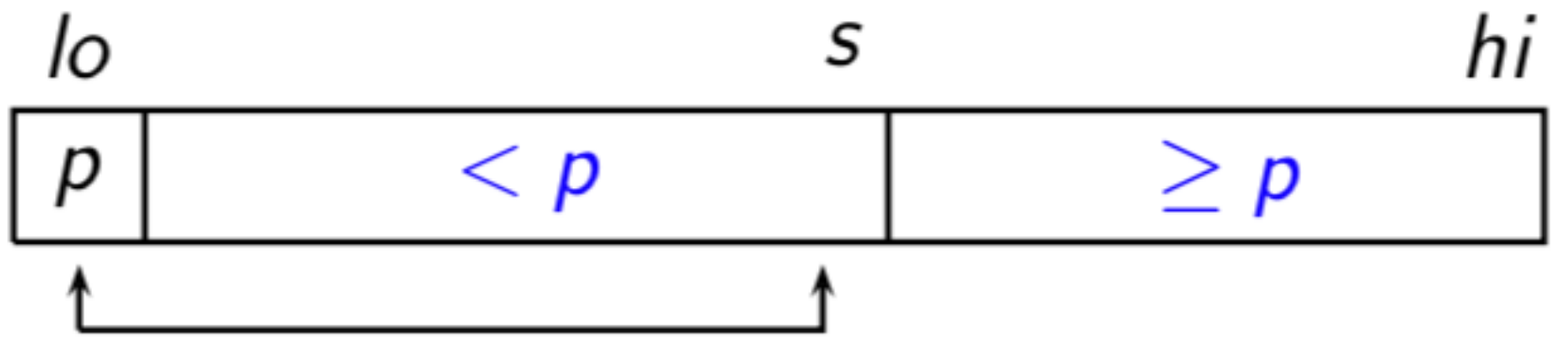
$s \leftarrow s + 1$

$swap(A[s], A[i])$

$swap(A[lo], A[s])$

return s





Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Lomuto Partitioning

function LomutoPartition($A[\cdot], lo, hi$)

$p \leftarrow A[lo]$

$s \leftarrow lo$

for $i \leftarrow lo + 1$ **to** hi **do**

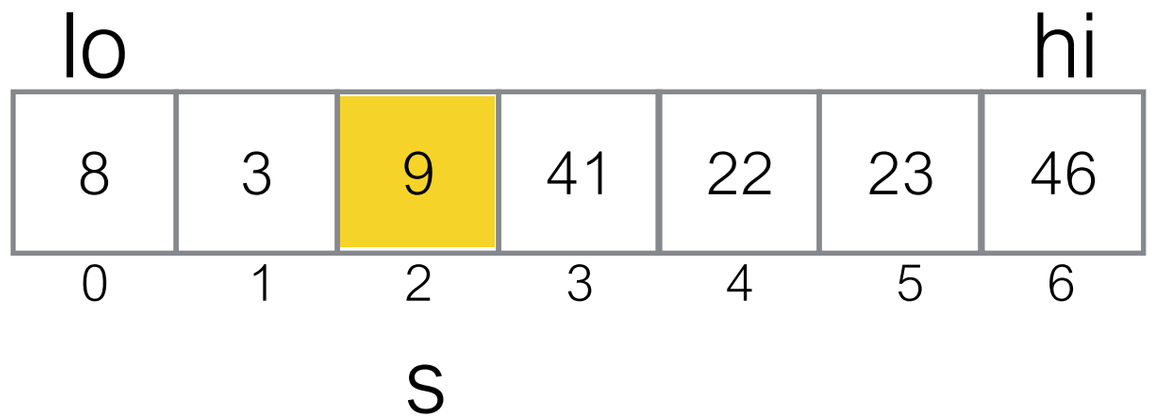
if $A[i] < p$ **then**

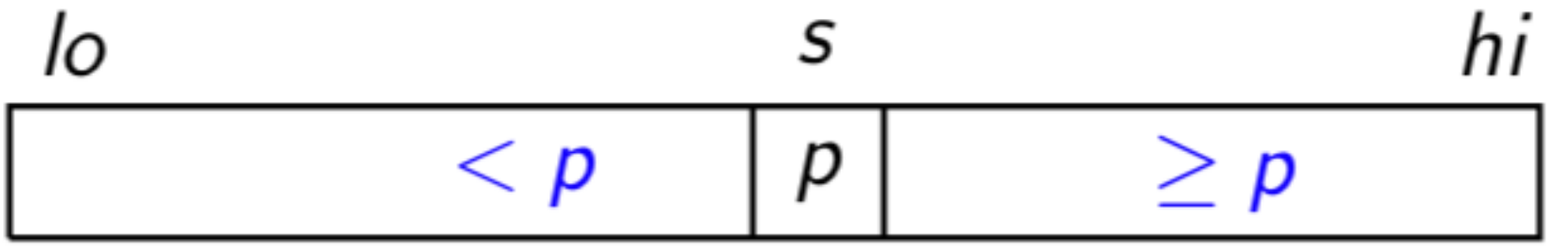
$s \leftarrow s + 1$

$swap(A[s], A[i])$

$swap(A[lo], A[s])$

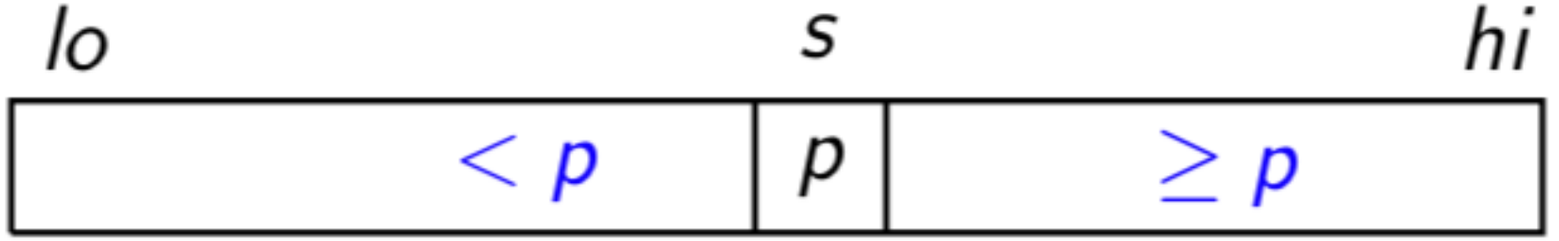
return s





Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Finding the *k*th-smallest Element



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Finding the *k*th-smallest Element



THE UNIVERSITY OF MELBOURNE

function QUICKSELECT($A[\cdot], lo, hi, k$)
    $s \leftarrow$ LOMUTOPARTITION(A, lo, hi)
    if $s - lo = k - 1$ **then**
        return $A[s]$
    else
    if $s - lo > k - 1$ **then**
        QUICKSELECT($A, lo, s - 1, k$)
    else
        QUICKSELECT($A, s + 1, hi, (k - 1) - (s - lo)$)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

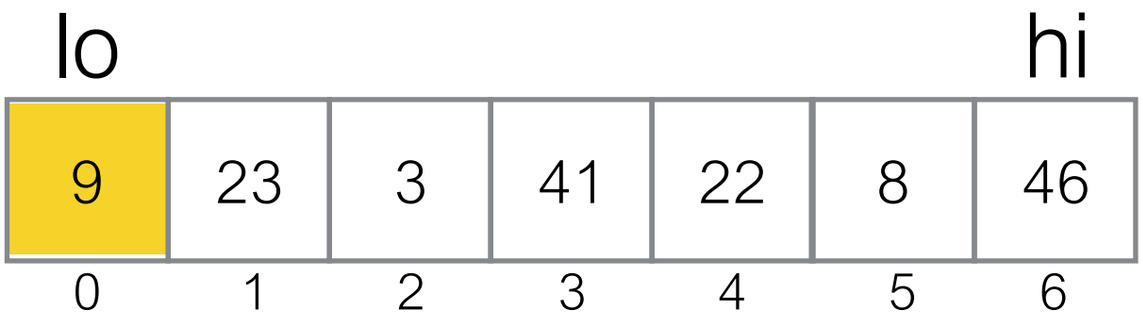
Example: Find the Sixth Smallest Element



THE UNIVERSITY OF MELBOURNE

```
function QuickSelect(A[·], lo, hi, k)
    s ← LomutoPartition(A, lo, hi)
    if s − lo = k − 1 then
        return A[s]
    else
        if s − lo > k − 1 then
            QuickSelect(A, lo, s − 1, k)
    else
        QuickSelect(A, s + 1, hi, (k − 1) − (s − lo))
```

k: 6



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

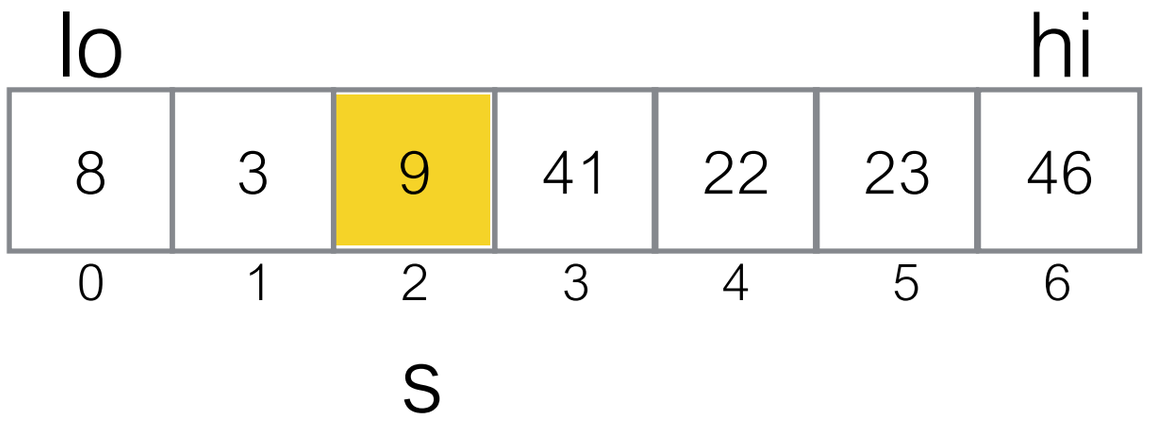
Example: Find the Sixth Smallest Element



THE UNIVERSITY OF MELBOURNE

```
function QuickSelect(A[·], lo, hi, k)
    s ← LomutoPartition(A, lo, hi)
    if s − lo = k − 1 then
        return A[s]
    else
        if s − lo > k − 1 then
            QuickSelect(A, lo, s − 1, k)
    else
        QuickSelect(A, s + 1, hi, (k − 1) − (s − lo))
```

k: 6



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

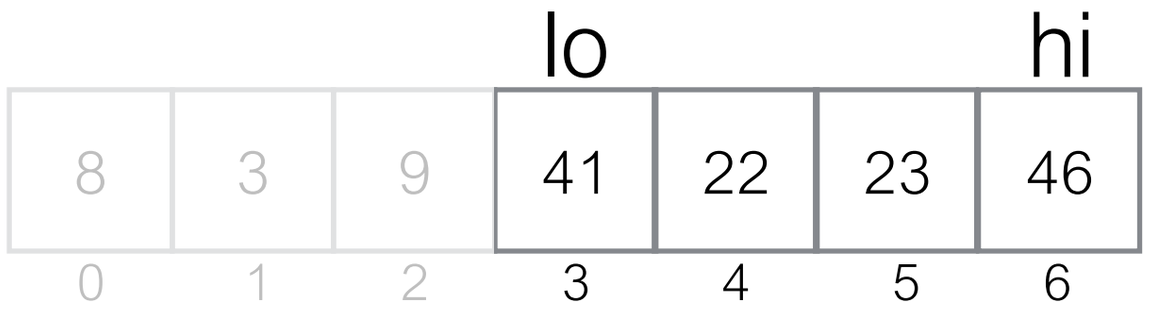
Example: Find the Sixth Smallest Element



THE UNIVERSITY OF MELBOURNE

```
function QuickSelect(A[·], lo, hi, k)
    s ← LomutoPartition(A, lo, hi)
    if s − lo = k − 1 then
        return A[s]
    else
        if s − lo > k − 1 then
            QuickSelect(A, lo, s − 1, k)
    else
        QuickSelect(A, s + 1, hi, (k − 1) − (s − lo))
```

k: 3



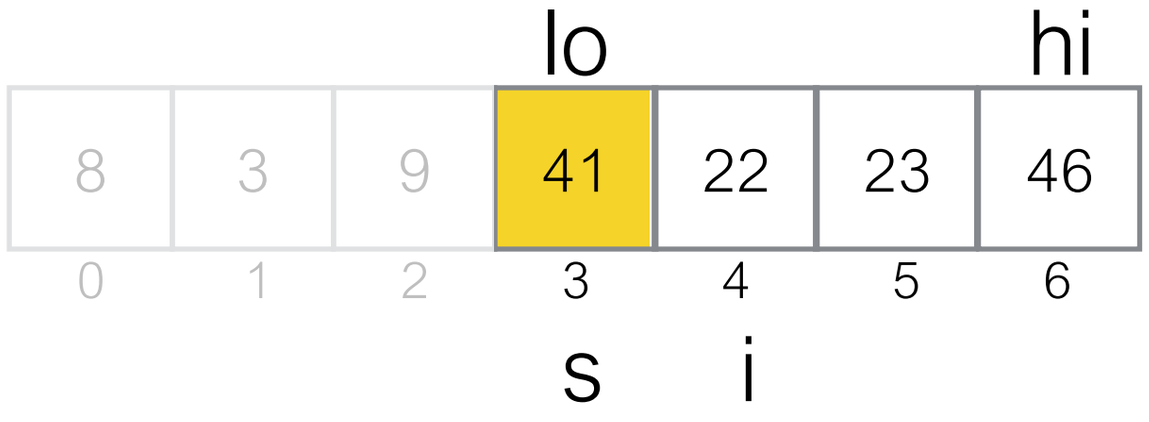
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Find the Sixth Smallest Element



THE UNIVERSITY OF MELBOURNE

```
function LomutoPartition(A[·], lo, hi)
    p ← A[lo]
    s ← lo
    for i ← lo + 1 to hi do
        if A[i] < p then
            s ← s + 1
            swap(A[s], A[i])
    swap(A[lo], A[s])
    return s
```



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Find the Sixth Smallest Element



THE UNIVERSITY OF MELBOURNE

function LomutoPartition($A[\cdot], lo, hi$)

$p \leftarrow A[lo]$

$s \leftarrow lo$

for $i \leftarrow lo + 1$ **to** hi **do**

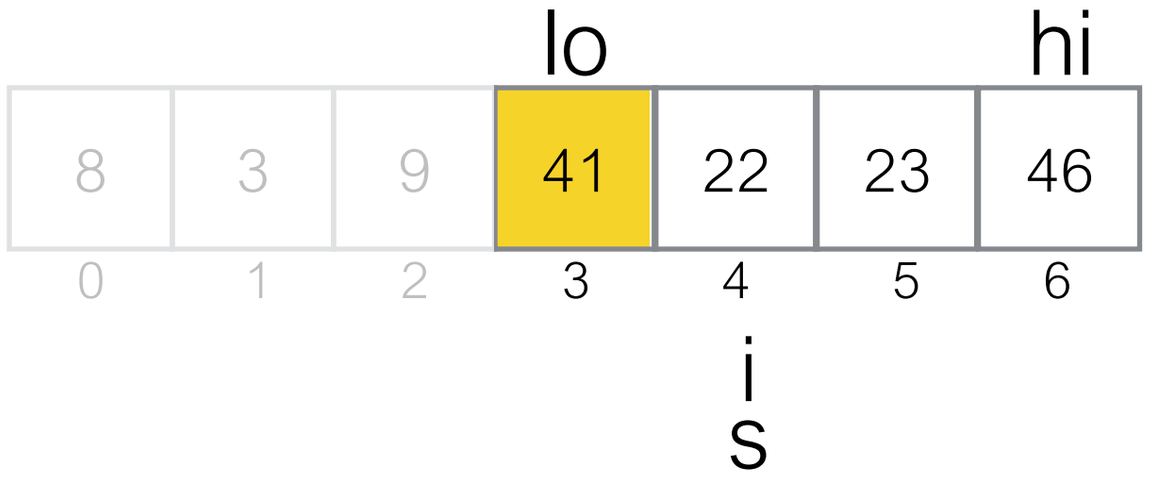
if $A[i] < p$ **then**

$s \leftarrow s + 1$

$swap(A[s], A[i])$

$swap(A[lo], A[s])$

return s

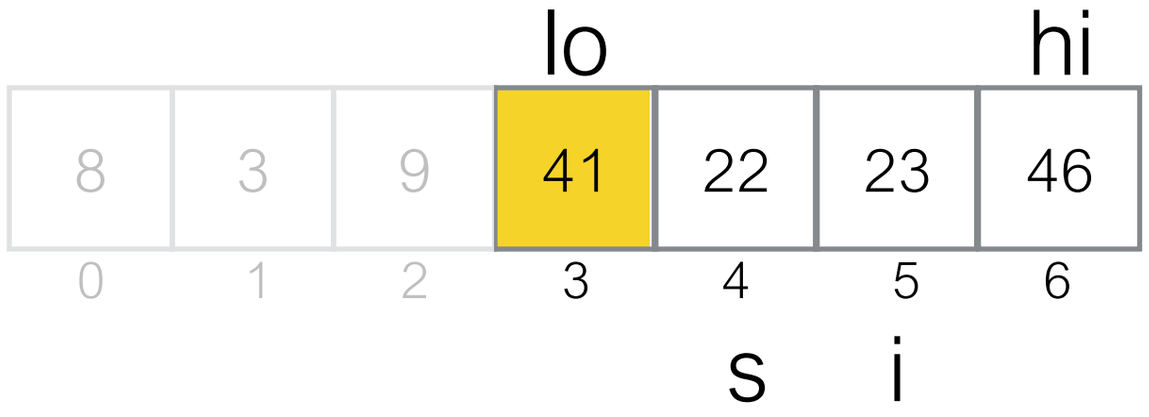


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Find the Sixth Smallest Element

THE UNIVERSITY OF MELBOURNE

```
function LomutoPartition(A[·], lo, hi)
    p ← A[lo]
    s ← lo
    for i ← lo + 1 to hi do
        if A[i] < p then
            s ← s + 1
            swap(A[s], A[i])
    swap(A[lo], A[s])
    return s
```



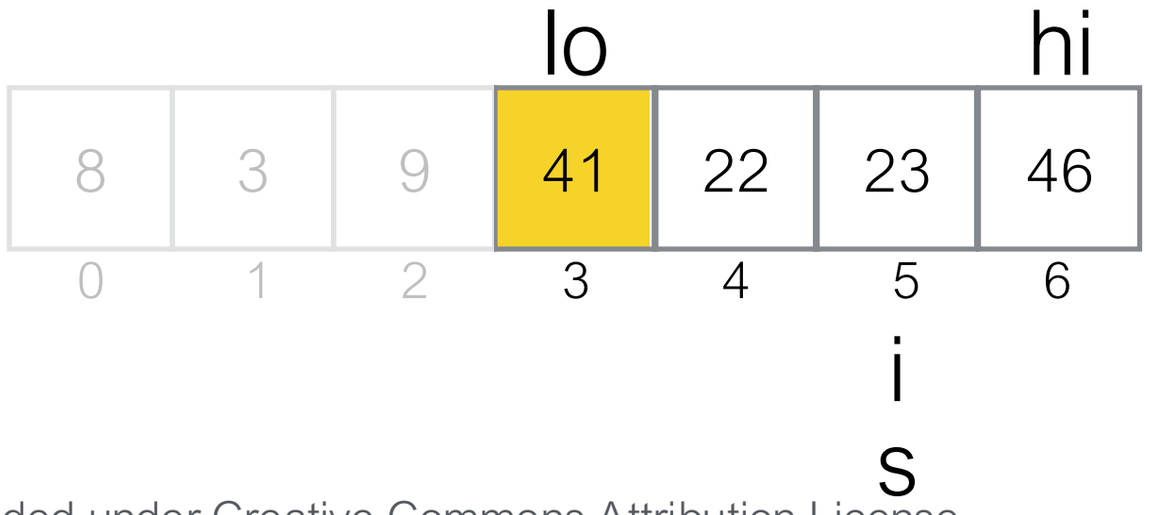
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Find the Sixth Smallest Element



THE UNIVERSITY OF MELBOURNE

```
function LomutoPartition(A[·], lo, hi)
    p ← A[lo]
    s ← lo
    for i ← lo + 1 to hi do
        if A[i] < p then
            s ← s + 1
            swap(A[s], A[i])
    swap(A[lo], A[s])
    return s
```



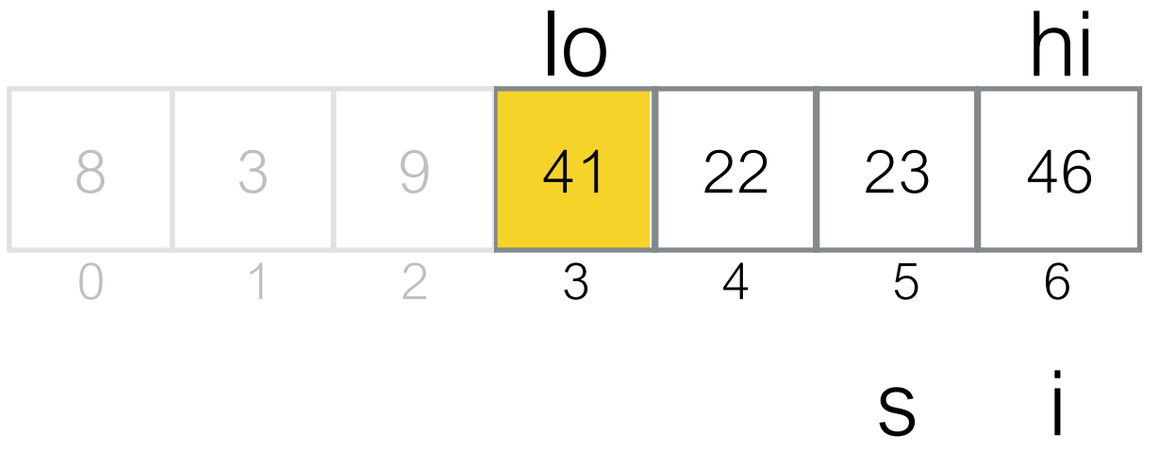
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Find the Sixth Smallest Element



THE UNIVERSITY OF MELBOURNE

```
function LomutoPartition(A[·], lo, hi)
    p ← A[lo]
    s ← lo
    for i ← lo + 1 to hi do
        if A[i] < p then
            s ← s + 1
            swap(A[s], A[i])
    swap(A[lo], A[s])
    return s
```

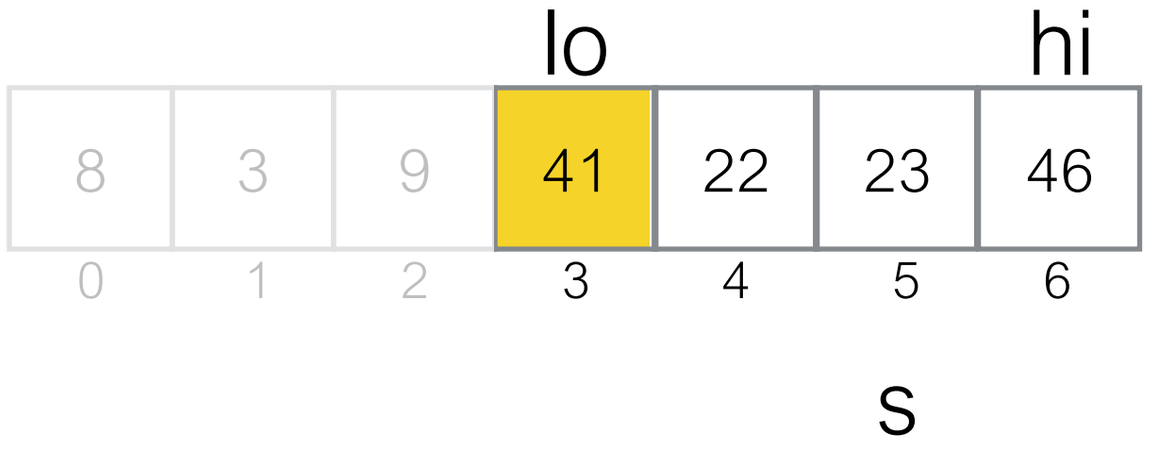


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Find the Sixth Smallest Element

THE UNIVERSITY OF MELBOURNE

```
function LomutoPartition(A[·], lo, hi)
    p ← A[lo]
    s ← lo
    for i ← lo + 1 to hi do
        if A[i] < p then
            s ← s + 1
            swap(A[s], A[i])
    swap(A[lo], A[s])
    return s
```



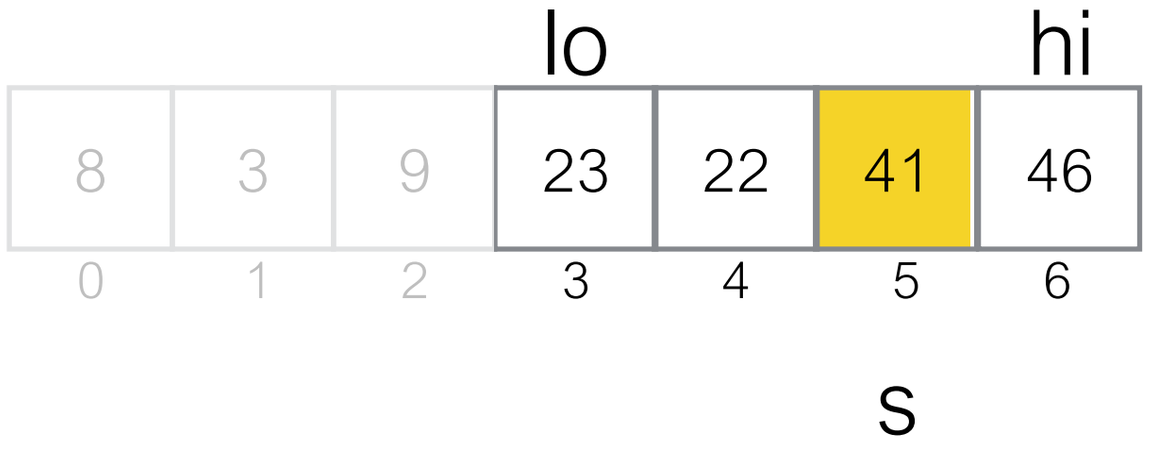
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Find the Sixth Smallest Element



THE UNIVERSITY OF MELBOURNE

```
function LomutoPartition(A[·], lo, hi)
    p ← A[lo]
    s ← lo
    for i ← lo + 1 to hi do
        if A[i] < p then
            s ← s + 1
            swap(A[s], A[i])
    swap(A[lo], A[s])
    return s
```



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

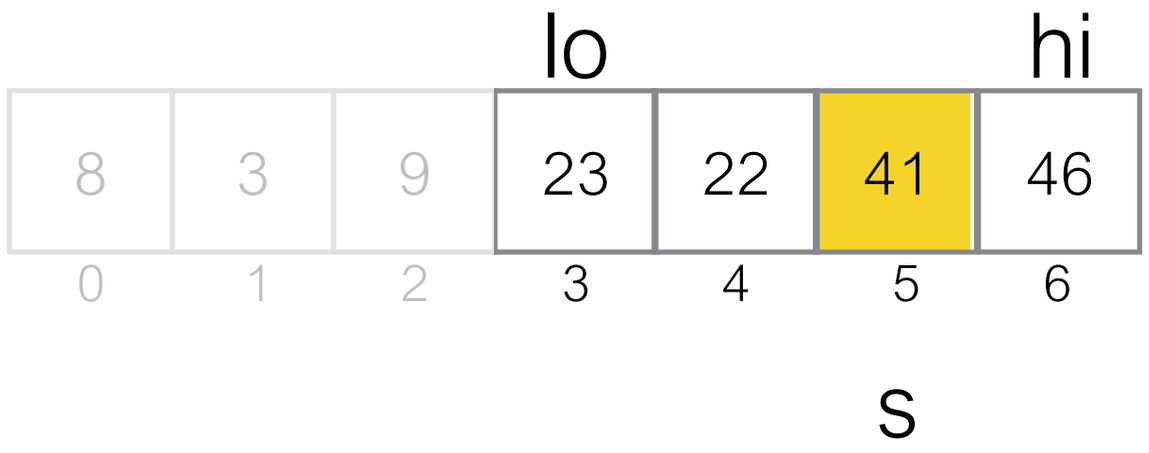
Example: Find the Sixth Smallest Element



THE UNIVERSITY OF MELBOURNE

```
function QuickSelect(A[·], lo, hi, k)
    s ← LomutoPartition(A, lo, hi)
    if s − lo = k − 1 then
        return A[s]
    else
        if s − lo > k − 1 then
            QuickSelect(A, lo, s − 1, k)
    else
        QuickSelect(A, s + 1, hi, (k − 1) − (s − lo))
```

k: 3



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Find the Sixth Smallest Element

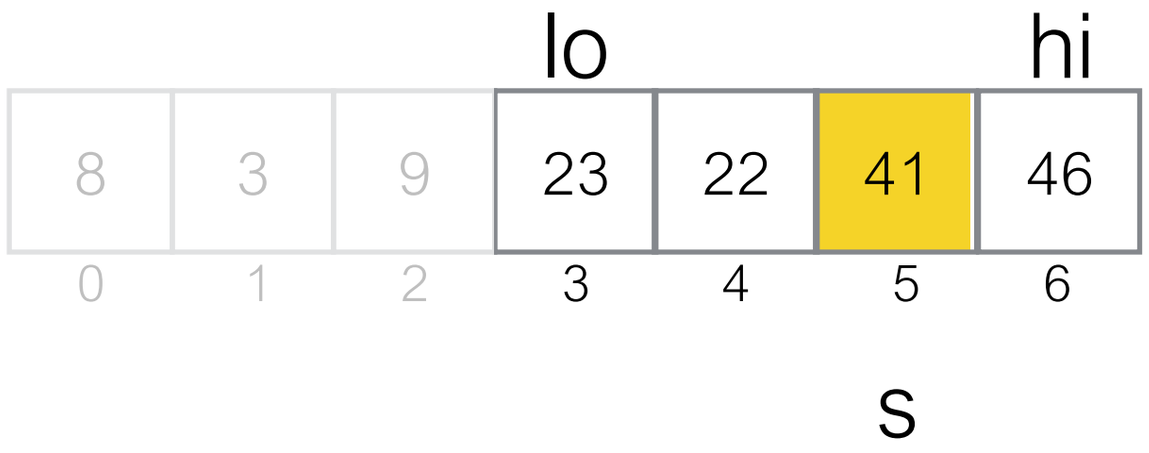


THE UNIVERSITY OF MELBOURNE

```
function QuickSelect(A[·], lo, hi, k)
    s ← LomutoPartition(A, lo, hi)
    if s − lo = k − 1 then
        return A[s]
    else
        if s − lo > k − 1 then
            QuickSelect(A, lo, s − 1, k)
    else
        QuickSelect(A, s + 1, hi, (k − 1) − (s − lo))
```

k: 3

returns 41!



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

QuickSelect Complexity

- **Worst case** complexity for QuickSelect is quadratic,
- **Average-case** complexity is linear.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Next week

- Learn to divide and conquer!

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Assignment 1 is out

See LMS for extra consultation hours

- In person and on Zoom
- Ed Discussion board (do not post answers)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Consultation hours

Tuesdays 2-4pm in person, CIS Student Hub

Thursdays 10-11 am Zoom, see Canvas for a link

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License